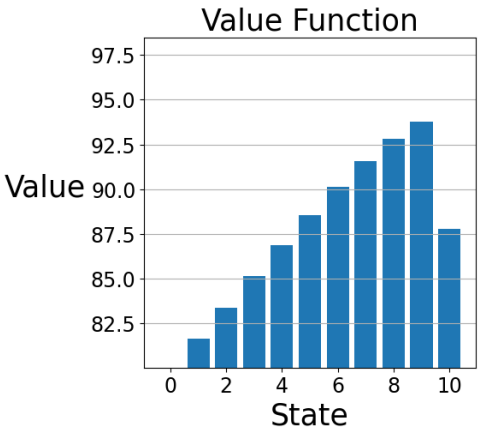
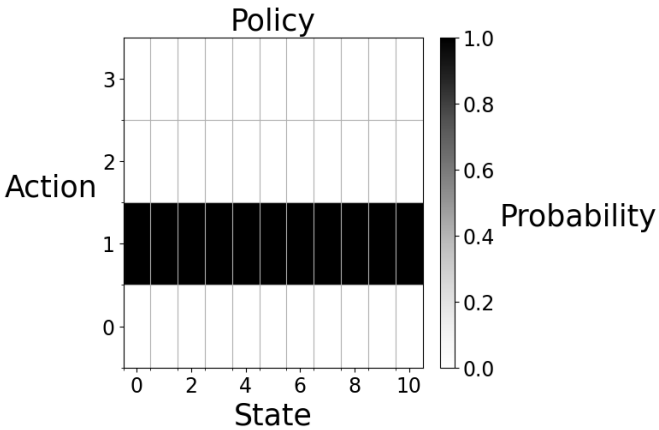
RL Lab 2 - Dynamic Programming

1. Policy evaluation

1.1. Please attach the Value function histogram and Action-State policy's probabilities heatmap obtained in this section (Section 1).

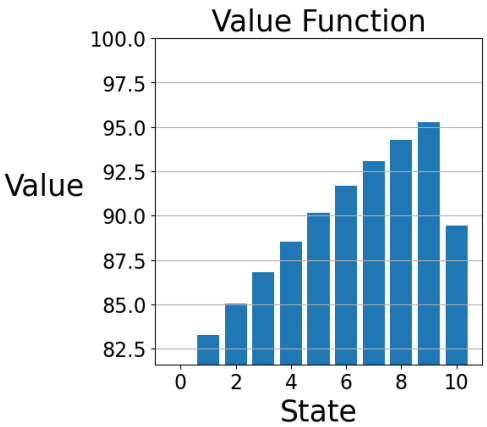


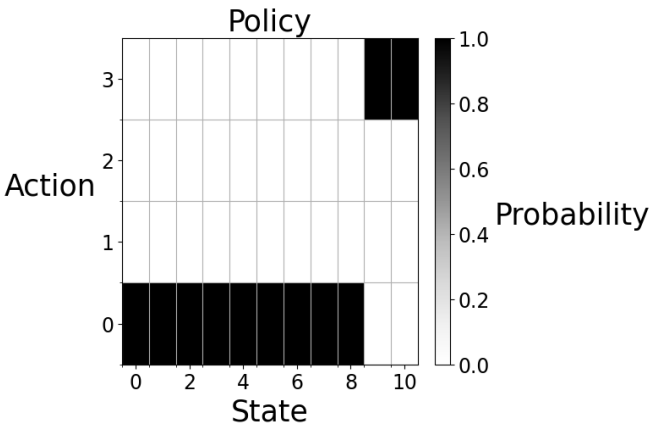


On the Value Function plot, we can see the value increases as the states rise, because the number of parking spots being filled increases wellfare. This is true until the slight drop at the end, which represents the fact the city doesn't want the parking spots to be completely full. The policy in this first case is static. The agent always prices the spot at 1, no matter the current state. This ist the best solution the agent came up with but it is counterintuitive to the theory as price should be adapted to encourage parking use when there are many empty slots and discourage it when all slots are full.

2: Policy Iteration

2.1. Please attach the Value function histogram and Action-State policy's probabilities heatmap obtained in this section (Section 2).

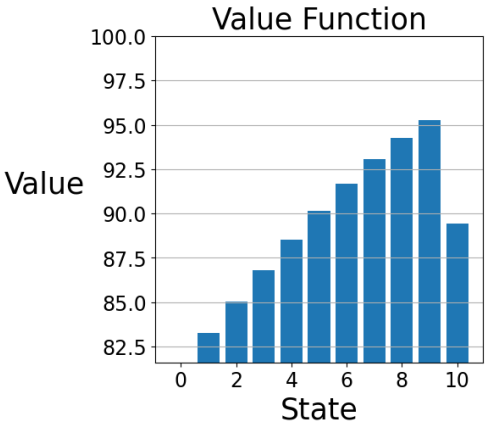


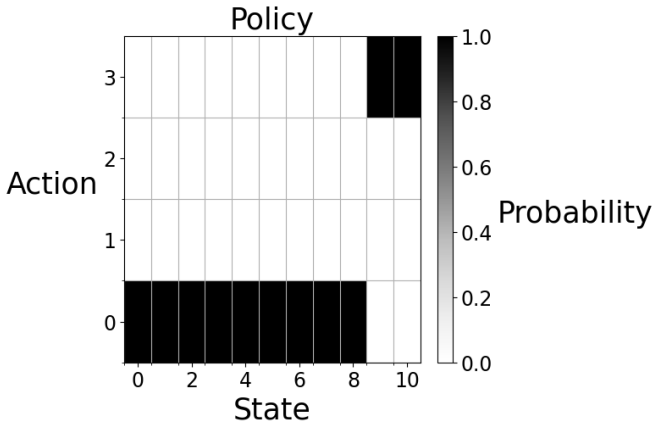


When adding policy iteration, we can see the shape of the value function graph is very similar to the previous one. However, the value at each state is higher. This is due tot he change in policy: when there are still many free spots, the city sets a low price (action 0) to encourage more parking. On the contrary, when the parking is almost full / full (states 9–10), the city sets a high price (action 3) to discourage more arrivals and avoid staying at "completely full". This is what is shown on the Policy graph.

3. Value Iteration

3.1. Please attach the Value function histogram and Action-State policy's probabilities heatmap obtained in this section (Section 3).





Using Value iteration instead of policy iteration leads you to the same intuitive result, but through different methods. Instead of picking a strategy, compute how good it is and adjusting it iteratively with policy iteration, value iteration assumes we act optimally from a point and update values accordingly.